

=====

Documento generado por MARU OS a partir de la base de conocimiento offline.
Fuentes etiquetadas honestamente (resúmenes MARU basados en normas/guias).
No sustituye asesoría médica, legal o de emergencia profesional.

1. Dengue: diagnóstico y manejo clínico (MINSA)

Fuente: NTS N° 125-MINSA/2016/CDC-DGIESP - Norma Técnica de Salud para la vigilancia y atención de casos de dengue

Agentes: aya, tupac

El dengue es una arbovirosis transmitida por el zancudo *Aedes aegypti*. Cuadro típico: fiebre alta (2-7 días), cefalea, dolor retroocular, mialgias, artralgias, erupción cutánea. Signos de alarma (requieren atención inmediata): dolor abdominal intenso y continuo, vómitos persistentes, sangrado de mucosas, letargia o irritabilidad, acumulación de líquidos, hepatomegalia >2 cm, aumento del hematocrito con caída rápida de plaquetas. REGLA CRÍTICA DE SEGURIDAD: en caso de dengue probable o confirmado está PROHIBIDO usar AINEs (ibuprofeno, naproxeno), aspirina (ácido acetilsalicílico) o corticoides, porque aumentan el riesgo de sangrado y gastritis hemorrágica. El único antipirético/analgésico indicado es PARACETAMOL (adultos: 500 mg - 1 g cada 6 horas, max. 4 g/día). No se usan antibióticos ni antivirales específicos. Manejo grupo A (sin signos de alarma): hidratación oral abundante, paracetamol, reposo, control diario y búsqueda activa de signos de alarma. Grupo B (con signos de alarma o comorbilidad): observación/hospitalización con hidratación endovenosa. Grupo C (dengue grave): emergencia - choque, sangrado grave o daño de órganos; manejo en UCI. Prevención: eliminar criaderos (recipientes con agua), uso de repelente y ropa de manga larga. Ante sospecha, acudir al establecimiento de salud más cercano; no automedicarse.

2. Atragantamiento: maniobra de Heimlich (compresiones abdominales)

Fuente: Guías AHA/Cruz Roja de primeros auxilios para obstrucción de la vía aérea por cuerpo extraño - Emergencias: SAMU

Agentes: aya, tupac

Obstrucción LEVE (la persona puede toser o hablar): animarla a seguir tosiendo con fuerza; no dar golpes ni maniobras todavía; vigilar de cerca. Obstrucción GRAVE en adulto o niño mayor de 1 año (no puede hablar, toser ni respirar; se lleva las manos al cuello): pedir que alguien llame al SAMU 106 y actuar de inmediato con la maniobra de Heimlich: colocarse detrás de la persona, rodear su cintura con los brazos, poner un puño con el pulgar hacia adentro justo por encima del ombligo (debajo del esternón), sujetar el puño con la otra mano y realizar compresiones bruscas hacia adentro y hacia arriba, repitiendo hasta que expulse el objeto o pierda el conocimiento. Alternativa recomendada: alternar 5 golpes secos entre los omoplatos con 5 compresiones abdominales. En gestantes o personas con obesidad: compresiones en el centro del pecho (torácicas), no abdominales. En bebés menores de 1 año: NO hacer Heimlich; sostener al bebé boca abajo sobre el antebrazo y dar 5 golpes entre los omoplatos, luego voltear y dar 5 compresiones torácicas con dos dedos, alternando. Si la persona queda inconsciente: bajarla al suelo con cuidado e iniciar RCP, revisando la boca antes de cada ventilación.

3. Ley N° 29733 de Protección de Datos Personales: datos de salud como sensibles

Fuente: Ley N° 29733, Ley de Protección de Datos Personales (2011) y su Reglamento (D.S. N° 003-2013-JUS; actualización)

Agentes: yaku, inti, kipu

La Ley N° 29733 desarrolla el derecho fundamental a la protección de los datos personales

(Constitucion, Art. 2 inc. 6). Se aplica a todo tratamiento de datos en bancos de datos publicos o privados. DATOS SENSIBLES: la ley califica como datos sensibles, entre otros, los datos de SALUD (fisica o mental), datos biometricos, origen racial y etnico, opiniones politicas, convicciones religiosas, y vida u orientacion sexual. Su tratamiento exige CONSENTIMIENTO previo, expreso, informado, inequivoco y POR ESCRITO (o medio equivalente) del titular, salvo excepciones legales (p. ej., atencion medica de emergencia o razones de salud publica previstas por ley). Principios rectores: legalidad, consentimiento, finalidad (los datos solo se usan para el fin informado), proporcionalidad, calidad, seguridad (medidas tecnicas y organizativas adecuadas) y disposicion de recurso. Derechos del titular (derechos 'ARCO'): Acceso a sus datos, Rectificacion, Cancelacion (supresion) y Oposicion al tratamiento. Se ejercen gratuitamente ante el titular del banco de datos; en caso de negativa, cabe reclamo ante la Autoridad Nacional de Proteccion de Datos Personales (Ministerio de Justicia) o proceso de habeas data. Implicancia practica para MARU OS: el perfil de salud del usuario (alergias, medicamentos, condiciones) es dato sensible - debe procesarse localmente, con consentimiento y sin compartirse con terceros.

4. Guia de formatos de modelos de IA: GGUF, PyTorch/Transformers, ONNX, Keras 3, Flax/JAX

Fuente: Documentacion oficial: llama.cpp/GGUF (ggml-org), Hugging Face Transformers, ONNX Runtime, Keras 3, Flax/JAX

Agentes: kipu

GGUF (llama.cpp / Ollama): formato binario de ggml para inferencia LOCAL en CPU/RAM (con offload opcional a GPU/Metal). Empaqueta pesos cuantizados (Q4_0, Q4_K_M, Q8_0...) y metadatos en un solo archivo. Ideal para laptops y equipos de consumo - es el formato que usa MARU OS con Ollama (los gemma4 locales son GGUF Q4). Regla practica: un modelo Q4 necesita ~0.6 GB de RAM por cada mil millones de parametros, mas el contexto. Transformers / PyTorch (Hugging Face): pesos en safetensors/PyTorch con precision completa (FP16/BF16). Es el formato de referencia para FINE-TUNING y entrenamiento en GPU (CUDA/ROCm) y para servir a escala con vLLM. Requiere tipicamente 2 GB de VRAM por cada mil millones de parametros en FP16. ONNX (ONNX Runtime): grafo portable e interoperable. Corre en NAVEGADOR (WebGPU/WASM con onnxruntime-web), en ANDROID/iOS (onnxruntime-mobile) y en C#/NET, C++ y Java. Ideal para desplegar el mismo modelo en muchas plataformas sin dependencias de Python. Keras 3: API de alto nivel MULTI-BACKEND - el mismo codigo corre sobre JAX, TensorFlow o PyTorch. Buena opcion didactica y para prototipos que luego migran de backend sin reescribir. Flax/JAX: libreria de redes neuronales sobre JAX (compilacion XLA, transformaciones jit/grad/vmap/pmap). Es el estandar para ENTRENAMIENTO MASIVO en TPU y clusters (asi se entrenaron los modelos Gemma de Google). Resumen por hardware: CPU/RAM local -> GGUF; GPU para fine-tuning/serving -> PyTorch+vLLM; navegador/Android/C# -> ONNX; multi-backend -> Keras 3; TPU/investigacion a gran escala -> Flax/JAX.

5. Buenas practicas de codigo limpio

Fuente: Robert C. Martin, 'Clean Code' (2008); Andrew Hunt & David Thomas, 'The Pragmatic Programmer' (1999); PEP 8 (Python)

Agentes: kipu

Principios centrales del codigo limpio: NOMBRES: usar nombres que revelen intencion ('elapsed_days' mejor que 'd'); funciones con verbos, clases con sustantivos; evitar abreviaturas cripticas y comentarios que solo repiten el codigo - el mejor comentario es el que no hace falta. FUNCIONES: pequenas, que hagan UNA sola cosa y a un solo nivel de abstraccion; pocos parametros (idealmente ?3); sin efectos secundarios ocultos; preferir retornos tempranos a anidamientos profundos. DRY (Don't Repeat Yourself): cada pieza de conocimiento debe tener una representacion unica y autoritativa en el sistema - la

duplicacion multiplica los bugs. KISS (Keep It Simple): la solucion mas simple que funciona suele ser la correcta. YAGNI (You Aren't Gonna Need It): no construir funcionalidad especulativa 'por si acaso'. ERRORES: usar excepciones en lugar de codigos de retorno; no silenciar errores (nunca un 'except' vacio); fallar rapido y con mensajes utiles. PRUEBAS: tests automatizados como red de seguridad para refactorizar; regla F.I.R.S.T. - rapidos (Fast), independientes (Independent), repetibles (Repeatable), autoverificables (Self-validating), oportunos (Timely). La regla del Boy Scout: dejar el codigo un poco mas limpio de como lo encontraste. En Python, seguir PEP 8: snake_case para funciones/variables, PascalCase para clases, lineas 79-99 caracteres, imports ordenados (estandar, terceros, locales).

6. SOLID y patrones de disenio GoF esenciales

Fuente: Robert C. Martin - principios SOLID; Gamma, Helm, Johnson, Vlissides (GoF), 'Design Patterns: Elements of Reusable

Agentes: kipu

SOLID: S - Responsabilidad Unica: una clase debe tener una sola razon para cambiar. O - Abierto/Cerrado: abierto a extension, cerrado a modificacion (extender con nuevas clases, no editando las existentes). L - Sustitucion de Liskov: cualquier subclase debe poder usarse donde se espera la clase base sin romper el programa. I - Segregacion de Interfaces: mejor varias interfaces pequenas y especificas que una 'gorda' que obligue a implementar metodos innecesarios. D - Inversion de Dependencias: depender de abstracciones, no de implementaciones concretas (inyeccion de dependencias). Patrones GoF mas usados: CREACIONALES - Singleton: una unica instancia global (usar con moderacion; dificulta los tests). Factory Method / Abstract Factory: delegar la creacion de objetos para desacoplar del tipo concreto. Builder: construir objetos complejos paso a paso. ESTRUCTURALES - Adapter: envolver una interfaz incompatible para que encaje con la esperada. Decorator: anadir responsabilidades a un objeto dinamicamente sin herencia. Facade: fachada simple sobre un subsistema complejo. DE COMPORTAMIENTO - Observer: suscriptores notificados ante eventos (base de sistemas reactivos y UI). Strategy: intercambiar algoritmos en tiempo de ejecucion tras una interfaz comun. Command: encapsular una accion como objeto (deshacer/rehacer, colas). Consejo: los patrones resuelven problemas recurrentes, no se aplican por moda - primero el problema, luego el patron.

7. Estructuras de datos y algoritmos fundamentales

Fuente: Cormen, Leiserson, Rivest, Stein - 'Introduction to Algorithms' (CLRS, MIT Press); analisis asintotico estandar

Agentes: kipu

Estructuras fundamentales y sus costos tipicos (notacion Big-O): ARRAY/LISTA: acceso por indice $O(1)$; insercion/borrado en medio $O(n)$. LISTA ENLAZADA: insercion/borrado $O(1)$ con referencia al nodo; acceso $O(n)$. PILA (LIFO) y COLA (FIFO): push/pop y enqueue/dequeue $O(1)$. TABLA HASH (dict/map): insercion, busqueda y borrado $O(1)$ promedio, $O(n)$ en el peor caso por colisiones. ARBOL BINARIO DE BUSQUEDA balanceado (AVL, rojo-negro): busqueda/insercion/borrado $O(\log n)$. HEAP (cola de prioridad): minimo/maximo en $O(1)$, insercion y extraccion $O(\log n)$. GRAFO: representacion por lista de adyacencia (eficiente para grafos dispersos) o matriz de adyacencia. Algoritmos esenciales: BUSQUEDA BINARIA sobre datos ordenados: $O(\log n)$. ORDENAMIENTO - Quicksort: $O(n \log n)$ promedio, $O(n^2)$ peor caso; Mergesort: $O(n \log n)$ garantizado y estable, usa $O(n)$ extra; Heapsort: $O(n \log n)$ in-place. RECORRIDOS DE GRAFOS - BFS (anchura, con cola): camino mas corto en grafos sin pesos; DFS (profundidad, con pila/recursion): deteccion de ciclos, orden topologico. DIJKSTRA con heap: caminos minimos con pesos no negativos, $O((V+E) \log V)$. PROGRAMACION DINAMICA: dividir en

subproblemas superpuestos y memorizar resultados (Fibonacci, mochila/knapsack, distancia de edicion). Regla practica: primero elegir la estructura de datos correcta; el algoritmo suele seguir de forma natural.

8. Primeros auxilios basicos (hemorragia, quemadura, desmayo)

Fuente: Resumen MARU basado en principios de primeros auxilios AHA/Cruz Roja adoptados en capacitacion ciudadana

Agentes: tupac, aya

Hemorragia externa: presion directa con gasa limpia; elevar si es posible; no retirar apositos empapados - anadir encima. Quemaduras: agua tibia corriente; no remedios caseros grasos; cubrir con apositos limpios. Desmayo: tumbar, elevar piernas si no hay trauma; valorar via aerea y respiracion. Activar SAMU 106 ante compromiso vital. Este resumen no reemplaza curso formal de primeros auxilios.

9. Ollama y modelos locales en MARU

Fuente: Resumen MARU basado en documentacion publica de Ollama + arquitectura MARU OS

Agentes: kipu

MARU puede enrutar chat a modelos locales via Ollama (API HTTP). Formato habitual de pesos: GGUF/quantizations. La RAM disponible limita el tamano del modelo. Buenas practicas: verificar 'ollama list', probar un modelo pequeno primero, no exponer la API a redes no confiables. Si el modelo falla por VRAM/RAM, sugerir uno mas liviano o modo cloud solo si el usuario lo habilita.

10. RAG offline: boveda + base de conocimiento

Fuente: Resumen MARU de la arquitectura interna (knowledge_base + document_vault)

Agentes: kipu, inti

MARU combina: (1) base de conocimiento embebida offline con scoring lexico; (2) boveda documental del usuario (PDF/texto) con chunks; (3) opcionalmente Qdrant cuando el stack Docker esta arriba. Para desarrollo: preferir textos fieles y fuentes etiquetadas; no inventar citas. Si la API '/knowledge' esta reachable, el frontend puede refrescar el indice.